

Relazione tecnica - Configurazione e contesto di analisi

1. Scopo del documento

Questa relazione descrive in modo dettagliato la configurazione del progetto, la struttura dei componenti e il contesto di analisi. L'obiettivo è chiarire come vengono generati i dati, come transitano nel sistema e come vengono interpretati dagli script di monitoraggio. I problemi di sicurezza sono soltanto accennati, perché già documentati nel README.

2. Panoramica dell'architettura

Il progetto simula un ambiente bancario con tre blocchi principali:

1. **Server applicativo** (Flask) che riceve richieste HTTP e registra eventi.
2. **Flusso eventi** centralizzato in tempo reale (realtime_access.log).
3. **Monitor** che leggono lo stream e producono alert in blacklist.

La scelta architetturale evita input da database o log storici. Il monitoraggio si basa su un canale unico, coerente e in tempo reale, per simulare un contesto operativo dove le anomalie devono essere rilevate subito.

3. Struttura del workspace

Cartelle e file chiave:

- **server/** contiene il server Flask.
- **script/** contiene i monitor (problema_01 ... problema_10), l'orchestratore `run_problem.sh` e i generatori di test.
- **logs/realtime_access.log** è il flusso eventi in tempo reale.
- **blacklist.csv** contiene gli alert e lo storico di rischio.
- **clienti_banca.csv** contiene i dati anagrafici, email e 2FA.

4. Origine dei dati

4.1 Generazione degli eventi

Il server espone endpoint HTTP (login, bonifico, prelievo, deposito). Ogni chiamata genera un evento con i campi principali:

- timestamp
- customer_id
- ip
- azione
- importo
- iban

- session_duration

Questi eventi vengono:

1. salvati nel database locale per scopi storici;
2. scritti nel file `logs/realtime_access.log` per il monitoraggio in tempo reale.

4.2 Formato dello stream

Il flusso eventi usa un formato semplice e compatto per facilitare il parsing:

```
timestamp|customer_id|ip|azione|importo|iban|session_duration
```

Esempio:

```
2026-02-23T04:01:10|800001|192.168.10.12|LOGIN|||
```

Questo formato scelto per permettere l'uso di comandi standard (awk, cut) con separatore|' e per evitare overhead di parsing complesso.

5. Lettura e analisi dello stream

Gli script di monitoraggio eseguono:

- `tail -f logs/realtime_access.log`

Questo consente di leggere gli eventi nel momento in cui si verificano. Ogni script implementa:

- una finestra temporale di 60 secondi;
- una logica di soglia specifica;
- un output di alert in blacklist.

Il flusso non viene mai riletto da storico. L'analisi è esclusivamente streaming, con stato minimo in file temporanei quando serve contare eventi.

6. Blacklist e gestione del rischio

6.1 Formato blacklist

La blacklist è un CSV con intestazione fissa:

```
timestamp,tipo_elemento,elemento,azione_rilevata,gravita,recidivita,risk_score,stato,origine
```

6.2 Rischio cumulativo

La logica di rischio è centralizzata in `script/lib_blacklist.sh`:

- se un elemento è già presente, la recidivita' aumenta di 1;
- il risk_score si somma al valore precedente;
- quando il totale raggiunge 100, lo stato diventa BLOCKED.

Questo approccio modella escalation graduale e consente di individuare attività ripetute anche se singolarmente non critiche.

7. Orchestrazione e test

7.1 `run_problem.sh`

`run_problem.sh` coordina l'esecuzione di:

1. avvio del monitor in background;
2. avvio del test generator;
3. attesa e terminazione del monitor.

Lo script verifica anche che il server sia attivo, e in caso contrario lo avvia automaticamente.

7.2 Test generator

I generatori di test simulano attività specifiche per superare le soglie:

- inviano richieste con `curl`;
- scelgono account casuali dal CSV clienti;
- rispettano il pacing temporale richiesto.

Questi test garantiscono ripetibilità e validazione rapida del comportamento.

8. Contesto di sicurezza e motivazione

Il contesto simulato risponde a requisiti reali:

- **tempo reale**: le anomalie devono emergere subito;
- **basso rumore**: soglie semplici, facili da verificare;
- **tracciamento storico**: `risk_score` cumulativo e recidività;
- **azione immediata**: stato `BLOCKED` al superamento della soglia critica.

Il flusso eventi centralizzato favorisce una visione uniforme e riduce la dipendenza da sorgenti multiple non affidabili.

9. Accenno ai problemi monitorati

I problemi sono descritti nel README. In sintesi, i monitor coprono:

- pattern AML su bonifici;
- accessi simultanei;
- accessi notturni;
- subnet ATM inattesa;
- brute-force login;
- IP pubblici inattesi;
- pattern API ripetitivi;
- bonifici con importo zero;

- incoerenza ATM/bonifico;
- low & slow su importi piccoli.

Questi problemi sono stati scelti per coprire anomalie comportamentali, coerenza di rete e segnali a bassa intensità.

10. Limitazioni e assunzioni

- L analisi usa solo il flusso in tempo reale, non storico.
- Le soglie sono statiche e non adattive.
- Il contesto è simulato e non include traffico esterno reale.
- I test generator non rappresentano tutti gli edge case possibili.

Queste scelte semplificano il progetto e rendono chiaro il flusso di valutazione, ma non sostituiscono un sistema SIEM completo.

11. Conclusioni

La configurazione proposta fornisce un contesto coerente e realistico per studiare anomalie di sicurezza in tempo reale. La separazione netta tra stream eventi e output di alert consente di mantenere semplice il modello di analisi, pur introducendo un meccanismo di escalation con rischio cumulativo.

Il documento evidenzia dove nascono i dati, come vengono normalizzati nello stream, e come vengono trasformati in decisioni operative.

Allegato: comandi principali usati

- `tail -f` per lo stream
 - `awk -F` per parsing
 - `sort -u` e `wc -l` per conteggi
 - `grep -q` per match
 - `date -d` e `date +%H` per orari
 - `curl -s -G --data-urlencode` per i generatori
-

Fine documento.